Otimização do Sequenciamento na Produção de Baterias: Abordagem via RCPSP

Etapa 6: Resultados do Método Aproximado (Metaheurística)

Cristian Madeira (Pós), Otávio Novais (162962), Thaís Melo (164934)

Configuração dos experimentos

O método aproximado é o RKO (*Random-Key Optimizer*) [1], na implementação C++ dos autores, sem alterações, com um decodificador próprio para o RCPSP: o vetor de chaves gera uma lista de prioridade topologicamente válida (a cada passo, a atividade elegível de menor chave), convertida em cronograma pelo SSGS e melhorada pelo *forward-backward improvement* (FBI) [4]. O RKO executa nove metaheurísticas em paralelo (BRKGA, SA, GRASP, ILS, VNS, PSO, GA, LNS e BRKGA-CS), que compartilham um conjunto de soluções de elite, com os parâmetros padrão do framework. Foram usados dois critérios de parada:

- **Tempo:** 10 s por execução no J30 e 30 s no J60.
- **Orçamento de 50.000 cronogramas** (decodificações, somadas entre as threads), o critério usual da literatura [3]. A criação do conjunto inicial de soluções consome cerca de 10.800 cronogramas no J30 e 21.600 no J60, igual para todos os métodos.

Em ambos, a execução também termina ao atingir o ótimo conhecido. Cada instância foi resolvida 10 vezes, no mesmo notebook da Etapa 5 (Intel Core i5-1135G7, 16 GB). Às dez instâncias J30 da Etapa 5 somaram-se cinco J60 com $RS = 0{,}2$ e ótimo conhecido, também resolvidas pelo Gurobi com 600 s. A qualidade é medida pelo desvio relativo ao ótimo, $\text{RPD} = 100\,(C_{\max} - C^*)/C^*$.

Exato × aproximado

Tabela 1: Gurobi (Etapa 5) e RKO nas mesmas instâncias. RKO: melhor e médio $C_{\max}$ em 10 execuções e tempo médio por execução.

		Gurobi (600 s)		RKO, tempo (10 s / 30 s)			RKO, 50.000 cronogramas		
Instância	C^*	$C_{\max}$	Tempo (s)	Melhor	Médio	Tempo (s)	Melhor	Médio	Tempo (s)
j301_1	43	**43**	0,1	**43**	43,0	0,10	**43**	43,0	0,09
j309_2	92	**92**	286,5	**92**	92,0	0,12	**92**	92,0	0,11
j3013_1	58	59†	600,1	**58**	58,3	5,70	60	60,0	0,28
j3025_4	81	**81**	242,8	**81**	81,8	8,98	82	82,0	0,26
j3029_8	80	81†	600,1	81	81,0	10,20	82	82,0	0,33
j3037_7	92	**92**	40,9	**92**	92,0	0,11	**92**	92,0	0,10
j3041_10	99	**99**	165,7	**99**	99,0	0,14	**99**	99,0	0,12
j3045_6	129	**129**†	600,1	**129**	129,0	0,15	**129**	129,0	0,13
j3014_2	53	**53**	51,0	**53**	53,0	0,12	**53**	53,0	0,11
j3030_2	68	**68**	6,0	**68**	68,0	0,57	**68**	68,4	0,21
j605_4	72	77†	600,1	**72**	72,5	22,43	73	74,4	0,84
j609_4	87	92†	600,1	88	88,5	30,80	90	91,0	0,94
j6021_4	95	96†	600,1	**95**	95,0	3,71	98	99,0	0,74
j6037_2	95	96†	600,1	96	96,0	30,60	97	97,0	0,69
j6041_7	132	147†	600,1	134	134,0	30,71	137	138,5	0,84
RPD / tempo J30		0,30%	259,3	0,12%	0,28%	2,62	0,72%	0,78%	0,17
RPD / tempo J60		5,23%	600,1	0,74%	1,00%	23,65	2,78%	3,83%	0,81

Em negrito: igual ao ótimo C^*. †: limite de tempo atingido, sem prova de otimalidade. Últimas linhas: RPD médio de cada coluna de $C_{\max}$ e tempo médio, por conjunto.

J30. Os dois métodos chegam à mesma qualidade, mas o RKO é cerca de 100 vezes mais rápido: RPD médio de 0,28% em 2,6 s por execução, contra 0,30% do Gurobi em 259 s. Em j3013_1, o RKO atingiu o ótimo (58), enquanto o Gurobi terminou em 59. Em j3029_8, nenhum dos dois chegou ao ótimo (80). A vantagem do método exato é a prova de otimalidade, obtida em 7 das 10 instâncias.

J60. O Gurobi não provou a otimalidade de nenhuma instância em 600 s, com RPD médio de 5,2% (até 11,4% em j6041_7: 147 contra o ótimo 132). Com 30 s, o RKO obteve RPD médio de 1,0%, igual ou melhor que o Gurobi nas cinco instâncias e no ótimo em j605_4 e j6021_4. Mesmo limitado a 50.000 cronogramas (menos de 1 s), o RKO supera o Gurobi em 3 das 5 instâncias, com RPD médio de 3,8%. O ganho do método aproximado cresce com o tamanho, como esperado para a formulação tempo-indexada, cujo número de variáveis acompanha o produto entre atividades e horizonte.

Comparação entre estratégias de busca

Para avaliar a observação de que algoritmos genéticos têm dificuldade neste problema, o GA e o BRKGA [2] do mesmo framework foram executados isoladamente, com o mesmo decodificador, com e sem o FBI.

Tabela 2: RPD médio (%) nas 15 instâncias, 10 execuções cada; p: teste de Friedman entre os três métodos.

Critério de parada	Decodificador	GA	BRKGA	RKO	p
Tempo (10 s / 30 s)	SSGS + FBI	0,50	0,49	0,52	0,81
	Só SSGS	0,95	1,03	0,82	0,12
50.000 cronogramas	SSGS + FBI	1,71	1,81	1,80	0,27
	Só SSGS	4,26	4,84	4,43	0,04

Com o FBI, os três métodos são estatisticamente equivalentes nos dois critérios. A única diferença significativa aparece sem o FBI e com orçamento fixo: o GA supera o BRKGA (Wilcoxon com correção de Holm, $p = 0{,}02$), e o RKO fica entre os dois. A dificuldade esperada do GA, portanto, não se confirmou quando ele usa a mesma representação e o mesmo decodificador. Uma explicação provável para o desempenho do BRKGA é o orçamento: o framework divide os cronogramas entre nove cópias paralelas, e, com a população padrão de 987 indivíduos, cada cópia do BRKGA faz só 3 a 4 gerações, contra 8 a 11 do GA (população de 400). Os parâmetros não foram calibrados para esse orçamento, o que limita a conclusão. O fator de maior efeito é o decodificador: o FBI reduz o RPD médio de 1,8–2,1% para 0,7–0,8% no J30 e de 9,1–10,4% para 3,6–4,1% no J60 com orçamento fixo ($p < 0{,}001$ nos três métodos).

Síntese. Para instâncias pequenas, o modelo exato entrega o ótimo com prova, a um custo de minutos. Para instâncias maiores, o RKO encontra soluções melhores em segundos. O RKO será a metaheurística do artigo, com o GA e o BRKGA como referência de comparação.

Referências

[1] Chaves, A. A. et al. A random-key optimizer for combinatorial optimization. *Journal of Heuristics*, 31(4), 2025.

[2] Gonçalves, J. F.; Resende, M. G. C. Biased random-key genetic algorithms for combinatorial optimization. *Journal of Heuristics*, 17(5):487–525, 2011.

[3] Kolisch, R.; Hartmann, S. Experimental investigation of heuristics for resource-constrained project scheduling: an update. *European Journal of Operational Research*, 174(1):23–37, 2006.

[4] Valls, V.; Ballestín, F.; Quintanilla, S. Justification and RCPSP: a technique that pays. *European Journal of Operational Research*, 165(2):375–386, 2005.